



Figure 3: On the LEGO synthetic, 3-digit addition, and Natural Instructions, we identify examples of ordered skill sets in which training on a mixture of skills helps learn an individual skill faster than just training on that skill itself, given a fixed training budget.

find that stratified sampling on these skills only improves validation loss per skill by 0.007 points over random sampling on average (Figure 2 left), suggesting that utilizing skills does not improve model performance in this case.

- The empty graph demonstrates that each skill is independent. This can occur if skills are too granular; for instance, learning Spanish math problems is unlikely to help with English poem generation. This graph suggests that the best approach for learning an individual skill is to train on the skill itself. We see that empty graphs exist in real data; in Figure 2 (center), using data sources as skills on the Pile of Law [21] results in a nearly empty skills graph (3.9% dense).
- Graphs that are neither empty nor complete thus suggest a nontrivial order of how skill influence each other. *This is the setting in which we expect that identifying skills and exploiting their ordering will help the most.* In Figure 2 right, we use task categories, which capture broader reasoning patterns, as skills on Natural Instructions and find that the estimated graph has intermediate density (42.7% dense). We show concrete examples of how skills can be learned more efficiently on Natural Instructions in Section 2.2.

While these intuitive groupings result in ordered skill sets on some datasets (e.g., task categories on NI), this is not always the case (e.g., instruction types on Alpaca and sources on Pile of Law). Even though these groupings capture some notion of diversity in the dataset, our findings suggest that not just any semantic grouping induces an ordered skill set. We now empirically demonstrate that our definition of ordered skill sets aligns with how models learn and can be exploited for more data-efficient training.

## 2.2 Examples of skills and ordered skill sets

We provide examples of ordered skill sets on the LEGO synthetic dataset, an addition synthetic dataset, and subsets of the Natural Instructions dataset. On these datasets, we find that certain skills are better learned when trained along with their prerequisite skills rather than in isolation.

**LEGO skills** The LEGO synthetic, first introduced in [72], can evaluate a model’s ability to follow a chain of reasoning. In this synthetic, the letters of the alphabet,  $\mathcal{A}$ , are variables each with some binary label in  $\{0, 1\}$ . An individual sample consists of  $k$  clauses for some fixed  $k$  across the dataset, each of the form  $a = gx$  where  $a, x \in \mathcal{A}$  and  $g$  is either a negation (“not”) or assertion (“val”), e.g. we assign  $a$  to the value of  $x$ , or we assign  $a$  to the opposite label. At the end of the sentence, we prompt the model for what the value of one of these variables is. Two samples  $x \in \mathcal{X}$  are given below for  $k = 5$ :

Input:  $b = \text{not } y, r = \text{val } 1, m = \text{val } b, q = \text{val } m, y = \text{not } r$ . Output:  $b = 1$ .

Input:  $c = \text{val } x, p = \text{val } f, x = \text{val } k, f = \text{not } c, k = \text{val } 0$ . Output:  $k = 0$ .

These samples each correspond to a chain of reasoning; for instance the first sample has the chain  $r, y, b, m, q$ , where knowing  $q$ ’s label requires the most reasoning steps. We define the  $i$ th skill  $s_i$  as the model’s ability to know the  $i$ th variable of the chain. From our example above, the first sample belongs to  $\mathcal{X}_{s_3}$  and the second sample belongs to  $\mathcal{X}_{s_1}$ . To demonstrate the existence of ordered skill sets, we continually pre-train the 125M parameter GPT-Neo model [5, 13] over various mixtures of LEGO skills with  $k = 5$ . In Figure 3 (left), we find that in 35.9% fewer training steps, training on a balanced mixture of  $\mathcal{X}_{s_1}, \mathcal{X}_{s_2}$ , and  $\mathcal{X}_{s_3}$  resulted in the same validation loss of 0.01 as training solely on  $\mathcal{X}_{s_3}$ . This suggests that  $s_1, s_2$  helped unlock performance on  $s_3$  and that there exist edges from  $s_1$  or  $s_2$  to  $s_3$  in the skill graph. Additional observations are available in Appendix D.1, where we examine other edges as well as more complex reasoning chains, and the full skills graph corresponding to the ordered skill set for LEGO with  $k = 5$  is in Figure 10.

**Addition skills** We consider a variant of a synthetic 5-digit addition dataset analyzed in [44]. We show the existence of ordered skill sets for a simplified 3-digit addition dataset where we treat each digit prediction as a skill—the outputs, in this case, are the integers  $\{0, 1, \dots, 9\}$ . Examples are of the following form:

Input: A = 1 0 6 + 0 7 1 , A 0 = ? Output: 7    Input: A = 6 0 6 + 8 7 9 , A 2 = ? Output: 4

where ‘A 0’ refers to the ones digit of the output ( $s_1$ ) and ‘A 2’ refers to the hundreds digit ( $s_3$ ). In Figure 3 (center), we find that in 32% fewer training steps, training on a balanced mixture of  $\mathcal{X}_{s_1}$ , and  $\mathcal{X}_{s_2}$  resulted in the same validation loss of 0.01 as training solely on  $\mathcal{X}_{s_1}$ . That is, the ones digit addition skill can be improved by simultaneously learning the tens digit addition skill, even though the former should not require information from the latter—this is in line with observations from prior work that models do not always learn the ones digit addition first [44]. The full skills graph corresponding to the ordered skill set over 3-digit addition is in Figure 11.

**Natural Instructions (NI) skills** We show that ordered skill sets exist in NI [63] when we treat task categories as skills.

- In Figure 3 (top right), we show that ordered skill sets exist over crosslingual task categories. Training on Spanish question generation (QG) along with equal parts of English QG, Spanish question answering (QA), and English QA results in 4.1% lower validation loss than training only on Spanish QG. Remarkably, the former only uses 25% of the latter’s Spanish QG data. This suggests that there are edges from Spanish QA, English QA, and English QG to Spanish QG.
- In Figure 3 (bottom right), we see that training on the task category Text Matching along with Stance Detection helps decrease the loss on Stance Detection by 11%. This suggests that these categories, which both involve understanding the relationship between two input texts, share an edge.

The full skills graphs corresponding to the ordered skill sets over these task categories are in Figure 13. While equating task categories to skills may be noisy, these examples suggest that there is signal within real data that suggests that ordered skill sets can improve data efficiency.

### 2.3 Skill recovery

A final component of characterizing skills is unsupervised recovery of ordered skill sets. We consider embedding-based clustering approaches and a loss-based clustering approach for recovering LEGO skills. When clustering data using various trained and pre-trained embeddings, we find that they were unable to achieve above 39% accuracy on LEGO. Instead, we find that taking 10 random training runs and clustering data by their *loss* per timestep per run recovers the skills with 61% accuracy (Table 3). The intuition behind this method is that the validation losses on points from the same skill have similar trajectories as models learn. We discuss this approach more in Appendix D.2.

## 3 Skills-based data selection

Now that we have established the existence of ordered skill sets, we discuss how to use them for data selection. We state the data selection problem for learning across skills in Section 3.1. We discuss how to learn the skills graph that will be exploited in our data selection methods in Section 3.2. We then introduce two sampling methods that utilize the graph, a simple skill-stratified sampling method and the online sampling method SKILL-IT, in Section 3.3.

### 3.1 Problem statement

We are given an ordered training skill set  $\mathcal{S}_{\text{train}} = \{s_{\text{train},1}, \dots, s_{\text{train},k}\}$  on the training data, each with associated support set  $\mathcal{X}_{s_{\text{train},1}}, \dots, \mathcal{X}_{s_{\text{train},k}}$ , and an ordered evaluation skill set  $\mathcal{S}_{\text{eval}} = \{s_{\text{eval},1}, \dots, s_{\text{eval},m}\}$  of  $m$  evaluation skills on a separate evaluation dataset. We aim to select  $n$  samples from  $\mathcal{S}_{\text{train}}$  via a mixture of training skills,  $p \in \Delta^{k-1}$ , to achieve three goals depending on how  $\mathcal{S}_{\text{eval}}$  is constructed:

- **Continual pre-training:** when  $\mathcal{S}_{\text{eval}} = \mathcal{S}_{\text{train}}$ , our goal is select a mixture of training skills to learn all of them.
- **Fine-tuning:** when  $\mathcal{S}_{\text{eval}} \subset \mathcal{S}_{\text{train}}$ , our goal is to select a mixture of training skills to learn an individual target skill or subset of these skills.
- **Out-of-domain:** when  $\mathcal{S}_{\text{eval}} \cap \mathcal{S}_{\text{train}} = \emptyset$ , our goal is to select a mixture of training skills to learn a disjoint set of evaluation skills we cannot train on. This can arise when we have a separate downstream validation dataset or the skills identified in the training dataset are noisy.

Furthermore, we have a skills graph  $G = (\mathcal{S}_{\text{train}} \cup \mathcal{S}_{\text{eval}}, E)$ , where  $E \subseteq \mathcal{S}_{\text{train}} \times \mathcal{S}_{\text{eval}}$  and  $A \in \mathbb{R}^{k \times m}$  is a weighted adjacency submatrix, where  $A_{ij}$  describes the strength of the edge from  $s_{\text{train},i}$  to  $s_{\text{eval},j}$ . In Table 1, we summarize how the three different settings are constructed and how  $A$  varies across them. Next, we discuss how  $A$  can be estimated from the data.

### 3.2 Skills graph learning

The skills graph is important for determining how to sample from the ordered skill set for training efficiently. We present two approaches for learning the skills graph—brute-force and linear approximation. Algorithms are provided in Appendix B.2. By definition 2.2, the brute-force way of identifying edges involves fixing an overall training budget of  $H$  steps and 1)

| Setting                | $\mathcal{S}_{\text{eval}}$                                             | Skills graph                                                                                      |
|------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Continual pre-training | $\mathcal{S}_{\text{eval}} = \mathcal{S}_{\text{train}}$                | $A \in \mathbb{R}^{k \times k}$ , edges among all $\mathcal{S}_{\text{train}}$                    |
| Fine-tuning            | $\mathcal{S}_{\text{eval}} \subset \mathcal{S}_{\text{train}}$          | $A \in \mathbb{R}^{k \times m}$ , edges from all training skills to target skill subset           |
| Out-of-domain          | $\mathcal{S}_{\text{eval}} \cap \mathcal{S}_{\text{train}} = \emptyset$ | $A \in \mathbb{R}^{k \times m}$ , edges from all training skills to separate evaluation skill set |

Table 1: Summary of three settings—continual pre-training, fine-tuning, and out-of-domain. These settings are determined by how  $\mathcal{S}_{\text{eval}}$  is defined and result in different skills graphs used for our sampling methods.

---

**Algorithm 1: SKILL-IT Online Data Selection Algorithm**

---

- 1: **Input:** Ordered training skill set  $\mathcal{S}_{\text{train}}$ , ordered evaluation skill set  $\mathcal{S}_{\text{eval}}$ . Learning rate  $\eta$ ,  $T$  rounds,  $n$  samples,  $H$  training steps per run for graph learning, model  $f_1$ , window parameter  $w$ .
  - 2:  $A \leftarrow \text{LEARNGRAPH}(\mathcal{S}_{\text{train}}, \mathcal{S}_{\text{eval}}, H, f_1)$  (Alg. 2, 3).
  - 3: Initialize  $p_1^i = \exp(\eta \sum_{j=1}^m A_{ij})$  for all  $i \in [k]$ , the softmax over  $A$ .
  - 4: **for**  $t = 1, \dots, T - 1$  **do**
  - 5:   Observe losses  $L_{\text{eval},j}(f_t)$  for all  $s_{\text{eval},j} \in \mathcal{S}_{\text{eval}}$ .
  - 6:   Train model  $f_t$  with  $n/T$  samples from mixture  $p_t$  over  $\mathcal{S}_{\text{train}}$ . Update model  $f_{t+1} = \Phi(f_t, p_t)$ .
  - 7:   Set  $p_{t+1}^i = \exp(\eta \sum_{\tau=t-w+1}^t \sum_{j=1}^m A_{ij} L_{\text{eval},j}(f_\tau))$ .
  - 8: **end for**
- 

training and evaluating the model on each  $s_i$  and 2) training the model on each pair of  $(s_i, s_j)$  and evaluating on  $s_i$  and  $s_j$ . If the loss on  $s_j$  when trained on both  $s_i$  and  $s_j$  is lower, there exists an edge from  $s_i$  to  $s_j$ . This approach has runtime  $\mathcal{O}(Hk^2)$ , which is feasible for small  $k$ . When  $k$  is large, we can approximate this approach in linear time by training on each  $s_i$  for  $h < H$  steps and setting  $A_{ij} > 0$  if the loss on  $s_j$  decreases over  $h$  steps for a runtime of  $\mathcal{O}(hk)$ . This linear approach is necessary in the out-of-domain setting when  $\mathcal{S}_{\text{eval}}$  and  $\mathcal{S}_{\text{train}}$  are disjoint, as we do not train on data associated with  $\mathcal{S}_{\text{eval}}$ . In addition, both graph learning approaches can be performed on a smaller model, and the learned graph can be used for data selection for training a larger model (Appendix D.4).

### 3.3 Skills graph-aware sampling

We present two approaches for sampling over the mixture of training skills according to the skills graph: skill-stratified sampling, which samples uniformly over relevant training skills according to  $A$ , and SKILL-IT, which is an online generalization that incorporates knowledge of how skills are being learned throughout training.

#### 3.3.1 Skill-stratified sampling

A straightforward sampling approach is to discard training skills that do not benefit the evaluation skills and sample uniformly over the set of relevant training skills, which we call *skill-stratified sampling*. For continual pre-training, the relevant skills are the entire training skill set; for each  $s_{\text{train},i} \in \mathcal{S}_{\text{train}}$ ,  $\Pr(s_{\text{train},i}) = \frac{1}{k}$ . This enables each skill to have sufficient training data. For fine-tuning, the relevant skills are the target skills and prerequisite skills, which can be identified via positive entries of the  $i$ th column of  $A$  with  $\mathcal{S}_{\text{prereq}} = \{s_{\text{train},i} : \exists s_{\text{eval},j} \text{ s.t. } A_{ij} > 0\}$ . We then set  $\Pr(s) = \frac{1}{|\mathcal{S}_{\text{prereq}} \cup \mathcal{S}_{\text{eval}}|}$  for  $s \in \mathcal{S}_{\text{prereq}} \cup \mathcal{S}_{\text{eval}}$ . For the out-of-domain setting, skill-stratified sampling is over the set of prerequisite skills. For each  $s \in \mathcal{S}_{\text{prereq}}$ , we set  $\Pr(s) = \frac{1}{|\mathcal{S}_{\text{prereq}}|}$ . Next, we propose our online algorithm that exploits the graph dynamically for more efficient training.

#### 3.3.2 SKILL-IT online data selection algorithm

Despite accounting for prerequisite skills, one shortcoming of skill-stratified sampling is that even if a skill has already obtained sufficiently low validation loss early during training, we will continue to allocate the same weight to that skill throughout training. Therefore, we formulate our data selection problem as an online learning problem and propose SKILL-IT, which both prioritizes prerequisite skills and skills that are not yet learned.

We are given a budget of  $T$  rounds and  $n$  total samples to train on. At round  $t$ , we select a mixture  $p_t \in \Delta^{k-1}$  from the  $k$ -dimensional unit simplex, and for each training skill  $s_{\text{train},i} \in \mathcal{S}_{\text{train}}$ , we sample from  $\mathcal{X}_{s_{\text{train},i}}$  with proportion  $p_t^i$  for a total of  $\frac{n}{T}$  samples per round. Let  $f_t$  be the model at the start of round  $t$ . We can define  $f_t$  recursively as a function of the previous round’s model  $f_{t-1}$  and mixture  $p_{t-1}$  via a dynamics function  $\Phi : \mathcal{F} \times \Delta^{k-1} \rightarrow \mathcal{F}$ ; that is,  $f_t = \Phi(f_{t-1}, p_{t-1})$ . Let  $L_{\text{eval},j}(f_t)$  be the validation loss of  $f_t$  on  $s_{\text{eval},j}$ . Our goal is to select  $p_1, \dots, p_T$  to minimize loss per evaluation skill at